

ТЕХНОЛОГИЧНО УЧИЛИЩЕ ЕЛЕКТРОННИ СИСТЕМИ
към ТЕХНИЧЕСКИ УНИВЕРСИТЕТ - СОФИЯ

КУРСОВ ПРОЕКТ

Предмет: Операционни системи

Тема: Имплементиране на игра на бесеница

Ученик:

Александър Вазов

Научен ръководител:

Критиян Йочев

СОФИЯ

2026

1. Общо описание на системата

- **Предназначение:** Двупоточно мрежово приложение за игра на "Бесеница" между двама играчи през TCP протокол.
- **Архитектура:** Клиент-Сървър модел, разделен на две самостоятелни програми.
- **Модулност (Логика):** Разделение на мрежовата част от ядрото на играта - цялата логика (валидация на думи, следене на предположенията и състоянието на маскираната дума) е капсулирана в библиотеката game.h/game.c.
- **Технологичен стек:** C (стандарт C11), POSIX нишки (pthreads), TCP сокети, GNU Make.

2. Мрежов протокол за комуникация

2.1. Фаза 1: Handshake

- Спецификация на съобщенията при свързване и инициализация:
 - **Клиент → Сървър:** `<word>\n` (Тайната дума, която опонентът трябва да познае).
 - **Сървър → Клиент:** `OK\n` (Думата е валидна, изчакване на старт).
 - **Сървър → Клиент:** `ERROR\n` (Невалидна дума, прекратяване на връзката).

2.2. Фаза 2: Състояние на играта

- Описание на 3-редовия фрейм, който сървърът изпраща първоначално и след всяко предположение:
 - `WORD <masked>\n` (Позициите на буквите, напр. `d__`).
 - `INCORRECT <a, b, c или празно>\n` (Списък с грешните букви).
 - `STATUS ONGOING\n` или `STATUS SOLVED\n` (Текущ статус на играча).
- **Клиент → Сървър:** Изпращане на предположение под формата на една буква: `<letter>\n`.
- **Протоколът е строго request-response:** Сървърът винаги връща фрейм след всяко съобщение от клиента - включително при повторно или невалидно предположение, при което фреймът е идентичен с предишния (без промяна на прогреса).

2.3. Фаза 3: Краен резултат

- Описание на съобщението след приключване на играта и от двамата играчи:
 - `RESULT WIN\n / LOSE\n / TIE\n`
 - `MY_INCORRECT <letters>\n`
 - `OPP_INCORRECT <letters>\n`

3. Сървърна архитектура и имплементация

3.1. Стартиране и Инициализация

- Създаване на TCP сокет, обвързване (bind) към адрес `0.0.0.0:<port>` и преминаване в режим на слушане (listen).
- Задаване на опцията `SO_REUSEADDR` върху сокета, което позволява незабавно повторно стартиране на сървъра на същия порт, без да се изчаква изтичане на OS таймаут.
- Извеждане на статус съобщение в конзолата: `Listening on <port>....`

3.2. Обработка на връзките

- Последователен цикъл в главната нишка (main thread) за приемане на играчи.
- Валидация на думата на всеки играч чрез `secret_word_init_from_c_string()`.
- Механизъм за самовъзстановяване при грешки: при невалидна дума връзката се затваря, но броячът `i` не се увеличава - цикълът продължава и чака нов опит за свързване, без да консумира игрален слот.
- Спиране на слушащия сокет след успешното настаняване на точно 2-ма играчи, което предотвратява допълнителни връзки на OS ниво.
- Спиране на слушащия сокет след успешното настаняване на точно 2-ма играчи.

3.3. Подготовка на играта

- Кръстосано присвояване на тайните думи (Играч 0 получава думата на Играч 1 и обратно).
- Инициализация на синхронизационни примитиви: споделен мютекс (`pthread_mutex_t`) и условна променлива (`pthread_cond_t`).
- Създаване на две независими нишки (`threads`) - по една за всеки играч.

3.4. Логика на игралните нишки

- Изпращане на първоначалното маскирано състояние.
- **Конкурентен игрален цикъл:** (четене на буква → обработка чрез `secret_word_guess()` → връщане на нов фрейм).
- **Синхронизация при край на играта:** условната променлива и броячът `finish_count` се използват за изчакване на втория играч. Проверката се извършва в цикъл `while` (а не с `if`), тъй като POSIX допуска т.нар. `spurious wake-ups` — събуждане на нишката без реален сигнал, след което условието трябва да се провери отново.
- **Определяне на победител:** Сравняване на броя грешни опити чрез функцията `letter_set_size()`.
- Изпращане на финалния фрейм `RESULT` и затваряне на връзките.
- При прекъсване на връзка преди решаване на думата: нишката достига `finish` блока, `g.solved[idx]` остава 0, а другият играч автоматично печели (проверка `!g.solved[opp_idx]`).

4. Клиентска архитектура и логика

4.1. Стартиране и Свързване

- Парсване на аргументи от командния ред (`<host>`, `<port>`, `<opponent-word>`).
- Инициране на TCP връзка и изпращане на думата.
- Обработка на отговора от сървъра (`OK` за продължаване или `ERROR` за прекратяване).

4.2. Главен игрален цикъл

- Логика за четене на данни от сървъра и разделяне на логическите разклонения (`RESULT` срещу `WORD`).
- Визуализация на игралното поле в конзолата (`Word: ...`, `Incorrect guesses: ...`).
- Контрол на състоянието (`STATUS SOLVED` преминава в блокиращо изчакване на `RESULT`, докато `STATUS ONGOING` чете локален вход от `stdin`).
- Локално филтриране на празни редове в терминала, без натоварване на сървъра.

4.3. Финален екран

- Обработка на финалните данни за грешните букви на двамата играчи и извеждане на съобщение за победа, загуба или равенство.

5. Ключови архитектурни решения

- **Пълна асинхронност и липса на ходове:** Играчите не се изчакват. Нишките работят паралелно, което предотвратява блокирането на бърз играч от по-бавен.
- **Сигурност на данните:** Сървърът никога не изпраща тайната дума в прав текст по мрежата - изпраща се само маскираната версия. Текстът се пази в паметта на сървъра.
- **Тиха обработка на невалидни състояния:** Повторно въведени или невалидни букви се филтрират от `secret_word_guess()`. Сървърът просто препраща старото състояние, карайки клиента да попита отново, без да променя прогреса.
- **Справедливо оценяване:** Победителят се определя въз основа на **точност (брой грешки)**, а не на скорост. Това елиминира мрежовото закъснение (ping) като фактор за победа.

6. Известни ограничения

- Няма поддръжка на повторно свързване - ако играч прекъсне връзката, играта приключва.
- Няма таймаут - ако играч спре да изпраща предположения, нишката на сървъра блокира безкрайно.
- Сървърът поддържа само една игра на стартиране и се изключва след нейното приключване.

7. Съдържание

1. Общо описание на системата	2
2. Мрежов протокол за комуникация	2
2.1. Фаза 1: Handshake	2
2.2. Фаза 2: Състояние на играта	2
2.3. Фаза 3: Краен резултат	3
3. Сървърна архитектура и имплементация	3
3.1. Стартиране и Инициализация	3
3.2. Обработка на връзките	3
3.3. Подготовка на играта	3
3.4. Логика на игралните нишки	4
4. Клиентска архитектура и логика	4
4.1. Стартиране и Свързване	4
4.2. Главен игрален цикъл	4
4.3. Финален екран	5
5. Ключови архитектурни решения	5
6. Известни ограничения	5
7. Съдържание	6